Práctica 6: Controlador de Temperatura ON-OFF

Presenta:

Jesús Ángel Bonilla Duran

Viridiana Santibáñez Carrillo

Jocelyn Rodriguez Torres

Matricula:

24033407

24033066

24033436

Asignatura:

Colocar Nombre de la Asignatura

Profesor:

Hugo Antonio Méndez Guzmán

Silao, Gto. a 06 de noviembre de 2026

Contenido

Introducción ... 3
Marco Teórico ... 3
Sistemas de control automático ... 3
Control de lazo cerrado ... 3
Control ON-OFF con histéresis ... 4
Microcontrolador ESP32 ... 4
Sensor de temperatura LM35 ... 4
Teclado matricial 4×4 ... 5
Pantalla LCD 16×2 con comunicación I2C ... 5
Actuadores y salidas digitales ... 5
Desarrollo ... 6
Material requerido ... 6
Explicación del desarrollo: ... 6
Funcionamiento del código ... 7
Resultados ... 9
Observaciones y Conclusiones ... 12
Observaciones ... 12
Conclusiones ... 13
Referencias ... 13

Lista de Figuras

Figura 1.diagrama wokwi ... 6
Figura 2. Montaje general del circuito de control de temperatura. ... 10
Figura 3. Visualización de la temperatura y los límites configurados en la pantalla LCD. ... 10
Figura 4. Funcionamiento del sistema con los parámetros de temperatura establecidos. ... 10
Figura 5. Configuración del límite superior de temperatura mediante el teclado matricial. ... 11
Figura 6. Visualización de los límites de temperatura y lectura del sensor LM35. ... 11
Figura 7. Integración del teclado matricial con el sistema de control. ... 12
Figura 8. Conexión de los actuadores y elementos de alimentación del sistema. ... 12

Introducción

Los sistemas de control automático desempeñan un papel fundamental en la industria moderna al permitir regular variables físicas críticas como la temperatura mediante la integración de sensores, algoritmos de control y actuadores de potencia. En el ámbito de los sistemas embebidos, el uso de microcontroladores de altas prestaciones facilita el procesamiento en tiempo real de señales del proceso y la ejecución de leyes de control que garantizan la estabilidad y seguridad en aplicaciones térmicas.

En esta práctica integradora se desarrollará un sistema embebido basado en la tarjeta de desarrollo ESP32 para la implementación de un controlador de temperatura de tipo ON-OFF con histéresis. La adquisición de datos se realiza mediante un sensor digital DS18B20 sobre un bus 1-Wire, mientras que la interfaz de usuario integra un teclado matricial 4×4 para la configuración dinámica de los parámetros de referencia (setpoint e histéresis) y una pantalla LCD 16×2 con interfaz I2C para la visualización del estado del proceso.

El propósito central de la práctica es consolidar los conceptos fundamentales del control automático y la programación no bloqueante, analizando el comportamiento de la banda de histéresis en la prevención del desgaste de los elementos de potencia. Asimismo, se empleará el entorno PlatformIO en conjunto con el simulador Wokwi para la depuración, compilación y prueba funcional del sistema, vinculando la adquisición de variables físicas, la gestión de periféricos y el diseño de etapas de potencia en la ingeniería de sistemas embebidos.

Marco Teórico

Sistemas de control automático

Los sistemas de control automático se utilizan para regular el funcionamiento de diferentes procesos, como la temperatura, la velocidad o la presión. Para realizar estas tareas se utilizan sensores, controladores y actuadores que trabajan en conjunto para obtener el resultado deseado.

En un sistema de control de temperatura, el sensor se encarga de medir los cambios de temperatura, mientras que el controlador recibe esa información y determina qué acción debe realizarse. De esta manera, se puede mantener una temperatura dentro de ciertos límites sin tener que realizar los cambios manualmente (International Society of Automation [ISA], 2023).

Control de lazo cerrado

El control de lazo cerrado es un sistema en el que se utiliza la información de salida para tomar decisiones sobre el funcionamiento del proceso. Esto se logra mediante un sensor que mide constantemente la variable que se desea controlar y envía esa información al controlador.

En esta práctica se utiliza este tipo de control porque el ESP32 recibe las lecturas de temperatura del sensor LM35 y las compara con los límites establecidos. Dependiendo del resultado, el programa activa la calefacción o el enfriamiento. De esta manera, el sistema puede responder a los cambios de temperatura utilizando la información que obtiene del sensor (ISA, 2023).

Control ON-OFF con histéresis

El control ON-OFF es un método de control en el que las salidas solamente tienen dos estados: encendido o apagado. Se utiliza principalmente en sistemas donde no es necesario regular la potencia de manera gradual, sino activar o desactivar un elemento dependiendo de la condición del proceso.

La histéresis consiste en establecer una diferencia entre el límite inferior y el límite superior de operación. Esto evita que el sistema cambie constantemente de estado cuando la temperatura se encuentra cerca del valor de referencia. También ayuda a disminuir el desgaste de los elementos que se activan y desactivan durante el funcionamiento.

En esta práctica se establecen dos límites de temperatura. Cuando la temperatura alcanza o supera el límite superior (HL), se activa el enfriamiento y se desactiva la calefacción. Cuando la temperatura alcanza o baja del límite inferior (LL), se activa la calefacción y se desactiva el enfriamiento. Si la temperatura permanece entre ambos límites, el sistema conserva el estado anterior (ISA, 2023).

Microcontrolador ESP32

El ESP32 es un microcontrolador que permite desarrollar proyectos electrónicos mediante la conexión de sensores, pantallas, motores y otros dispositivos. Cuenta con entradas y salidas de propósito general (GPIO), comunicación I2C, comunicación serial y convertidores analógico-digitales, entre otras funciones.

En esta práctica, el ESP32 funciona como la parte principal del sistema, ya que recibe la información del sensor LM35, procesa los datos de temperatura, interpreta las teclas presionadas y controla las salidas digitales encargadas de activar los elementos de calefacción y enfriamiento (Espressif Systems, s. f.).

Sensor de temperatura LM35

El LM35 es un sensor de temperatura analógico que proporciona un voltaje proporcional a la temperatura que está midiendo. Una de sus características principales es que tiene una escala de salida de 10 milivoltios por cada grado Celsius, lo que permite convertir fácilmente el voltaje obtenido a una lectura de temperatura.

En esta práctica, el LM35 se conecta a una entrada analógica del ESP32, específicamente al GPIO 32. El microcontrolador recibe la señal eléctrica del sensor y utiliza su convertidor analógico-digital (ADC) para transformar esa señal en un valor que pueda procesar el

programa. Posteriormente, el resultado se convierte a grados Celsius para realizar el control de temperatura (Texas Instruments, 2017; Espressif Systems, s. f.).

Teclado matricial 4×4

El teclado matricial 4×4 es un dispositivo de entrada que cuenta con 16 teclas distribuidas en cuatro filas y cuatro columnas. Su funcionamiento se basa en identificar la combinación de fila y columna que se produce cuando se presiona una tecla.

Este tipo de teclado permite reducir la cantidad de conexiones necesarias con el microcontrolador, ya que no requiere utilizar un pin independiente para cada botón. El ESP32 puede identificar las teclas mediante un proceso de lectura de filas y columnas.

En esta práctica, el teclado se utiliza para ingresar y modificar los límites de temperatura del sistema. También permite guardar los valores introducidos o cancelar la configuración cuando sea necesario (Microchip Technology Inc., s. f.).

Pantalla LCD 16×2 con comunicación I2C

La pantalla LCD 16×2 es un dispositivo que permite mostrar información mediante dos filas con capacidad para 16 caracteres cada una. Se utiliza en diferentes proyectos electrónicos para visualizar datos, mensajes y estados de funcionamiento.

Para facilitar su conexión con el ESP32 se utiliza el protocolo I2C, el cual permite la comunicación entre dispositivos mediante dos líneas principales: SDA, encargada de transmitir los datos, y SCL, encargada de proporcionar la señal de reloj.

En esta práctica, la pantalla LCD permite visualizar la temperatura registrada por el sensor LM35, los límites inferior y superior establecidos y el estado de los actuadores. Esto facilita conocer el funcionamiento del sistema durante las pruebas (Arduino, s. f.).

Actuadores y salidas digitales

Los actuadores son elementos que permiten realizar una acción dentro de un sistema de control. Pueden utilizarse para generar movimiento, calor, enfriamiento o cualquier otra acción necesaria para el funcionamiento de un proceso.

Las salidas digitales del microcontrolador permiten enviar señales de encendido o apagado para controlar diferentes dispositivos. En esta práctica, el ESP32 utiliza dos salidas digitales: una para la calefacción y otra para el enfriamiento. El programa determina cuál debe activarse de acuerdo con la temperatura registrada y los límites configurados.

De esta manera, los actuadores responden a las instrucciones del microcontrolador y permiten realizar el control automático de temperatura (ISA, 2023; Espressif Systems, s. f.).

Desarrollo

Material requerido

- ESP32 DevKit V1
- Keypad matricial 4×4
- LCD 16×2 con módulo I2C
- Sensor de Temperatura LM35
- LED o módulo de relevador
- PlatformIO
- Wokwi

Explicación del desarrollo:

Para el desarrollo de la práctica se realizó la configuración del circuito de control de temperatura en la herramienta Wokwi, utilizando un ESP32 como unidad de procesamiento, un sensor LM35 para medir la temperatura, un teclado matricial 4x4 para ingresar los límites de operación, una pantalla LCD I2C para visualizar la información y dos salidas digitales para controlar los actuadores de calefacción y enfriamiento, se establecieron las conexiones correspondientes a cada componente, utilizando el GPIO 32 para la lectura del sensor, los GPIO 19 y 23 para los actuadores, los GPIO 21 y 22 para la comunicación I2C y los pines GPIO 18, 33, 25, 26, 27, 14, 15 y 4 para el teclado matricial.

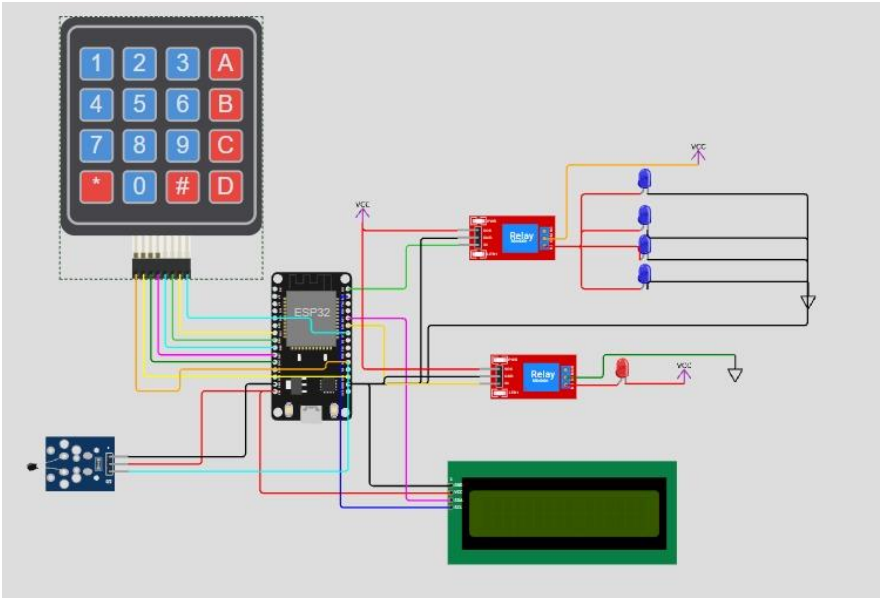


Figura 1.diagrama wokwi

Funcionamiento del código

El programa se desarrolló en lenguaje C++ mediante Arduino Framework, incorporando las funciones necesarias para la lectura del sensor, el ingreso y validación de los límites de temperatura y el control de los actuadores. Mediante el teclado matricial es posible configurar el límite inferior (LL) y el límite superior (HL), utilizando las teclas correspondientes para ingresar valores, guardar los cambios o cancelar la configuración para ello se establecieron condiciones para evitar valores superiores a 80 °C y garantizar que el límite inferior siempre sea menor que el superior.

Para obtener la temperatura se programó una función de lectura del sensor LM35, en la cual se toman 15 muestras de la señal analógica y se calcula su promedio, este valor se convierte a milivoltios y posteriormente a grados Celsius, utilizando la resolución de 12 bits del convertidor analógico-digital del ESP32, la lectura y actualización de esta variable se realizan periódicamente cada 375 ms.

El control de temperatura se implementó mediante una lógica de histéresis que compara continuamente la temperatura medida con los límites establecidos, es decir que cuando la temperatura alcanza o supera el límite superior, se activa el sistema de enfriamiento y se desactiva la calefacción; mientras que, cuando alcanza o desciende por debajo del límite inferior, se activa la calefacción y se desactiva el enfriamiento, si la temperatura permanece entre ambos límites, el sistema conserva el estado de operación anterior.

Finalmente, se incorporó la actualización de la pantalla LCD cada 150 ms, mostrando los valores configurados para los límites inferior y superior, la temperatura registrada por el sensor y el estado de los actuadores, identificado mediante los indicadores de calefacción [CALEN] y enfriamiento [ENFR.]. También se utilizó el monitor serial con una velocidad de comunicación de 115200 baudios para mostrar mensajes relacionados con el inicio del sistema, la configuración de límites y la validación de los valores ingresados.

A continuación, se muestra el código empleado en el archivo `main.cpp`:

main.cpp

```
#include <Arduino.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>
#define PIN_LM35 32
#define FOCO_CALENTAR 19
#define ENFRIAMIENTO 23

const byte ROWS = 4;
const byte COLS = 4;

char keys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};
byte rowPins[ROWS] = {18, 33, 25, 26};
byte colPins[COLS] = {27, 14, 15, 4};

Keypad keypad = Keypad(
  makeKeymap(keys),
  rowPins,
  colPins,
  ROWS,
  COLS
);
int HL = 35;
```

```
byte modoConfig = 0;
int valorTemporal = 0;
byte digitosIngresados = 0;

float temperatura = 25.0;
bool enfriando = false;

unsigned long tiempoLectura = 0;
unsigned long tiempoLCD = 0;

LiquidCrystal_I2C lcd(0x27, 16, 2);
float leerLM35() {
  long sumaADC = 0;
  const int MUESTRAS = 15;

  for (int i = 0; i < MUESTRAS; i++) {
    sumaADC += analogRead(PIN_LM35);
    delayMicroseconds(50);
  }

  float adcPromedio = (float)sumaADC / MUESTRAS;
  float milivoltios = (adcPromedio / 4095.0) * 3300.0;

  return milivoltios / 10.0;
}

void iniciarConfiguracion(byte modo) {
  modoConfig = modo;
  valorTemporal = 0;
  digitosIngresados = 0;

  Serial.println(modo == 1 ?
    "Configurando LL" :
    "Configurando HL");
}
void guardarConfiguracion() {
  if (digitosIngresados == 0) {
    Serial.println("ERROR: No ingresaste ningun valor");
    return;
  }
  if (valorTemporal > 80) {
    Serial.println("ERROR: Maximo permitido 80 C");
    return;
  }
  if (modoConfig == 1) {
    if (valorTemporal < HL) {
      LL = valorTemporal;
      Serial.print("Nuevo LL: ");
      Serial.println(LL);
      modoConfig = 0;
    }

    else {
      Serial.println("ERROR: LL debe ser menor que HL");
    }
  }

  else if (modoConfig == 2) {
    if (valorTemporal > LL) {
      HL = valorTemporal;
      Serial.print("Nuevo HL: ");
      Serial.println(HL);
      modoConfig = 0;
    }
    else {
      Serial.println("ERROR: HL debe ser mayor que LL");
    }
  }
}
void setup() {

  Serial.begin(115200);

  pinMode(FOCO_CALENTAR, OUTPUT);
  pinMode(ENFRIAMIENTO, OUTPUT);

  digitalWrite(FOCO_CALENTAR, HIGH);
  digitalWrite(ENFRIAMIENTO, HIGH);

  Wire.begin(21, 22);

  lcd.init();
  lcd.backlight();
  lcd.clear();

  analogReadResolution(12);

  Serial.println("Control de temperatura iniciado");
}
void loop() {

  char key = keypad.getKey();

  if (key != NO_KEY) {
    if (key == '*') {
      iniciarConfiguracion(1);
    }
    else if (key == 'B') {
      iniciarConfiguracion(2);
    }
    else if (key == 'C') {
      modoConfig = 0;
      Serial.println("Configuracion cancelada");
    }
    else if (key == 'D') {
      if (modoConfig != 0) {
        guardarConfiguracion();
      }
```

```
    }
    else if (key >= '0' && key <= '9' && modoConfig != 0) {
      if (digitosIngresados < 2) {
        int candidato = (valorTemporal * 10) + (key - '0');
        if (candidato <= 80) {
          valorTemporal = candidato;
          digitosIngresados++;

          Serial.print("Valor temporal: ");
          Serial.println(valorTemporal);

        }
        else {
          Serial.println("ERROR: Temperatura maxima 80 C");
        }
      }
    }
  }
  if (millis() - tiempoLectura >= 375) {
    tiempoLectura = millis();
    temperatura = leerLM35();
    if (temperatura >= HL) {
      enfriando = true;
    }
    else if (temperatura <= LL) {
      enfriando = false;
    }

    if (enfriando) {
      digitalWrite(FOCO_CALENTAR, HIGH);
      digitalWrite(ENFRIAMIENTO, LOW);
    }
    else {
      digitalWrite(ENFRIAMIENTO, HIGH);
      digitalWrite(FOCO_CALENTAR, LOW);
    }
  }
  if (millis() - tiempoLCD >= 150) {
    tiempoLCD = millis();
    char buffer[12];
    lcd.setCursor(0, 0);
    if (modoConfig == 1) {
      snprintf(buffer, sizeof(buffer), "LL=%02d_  ", valorTemporal);
    }
    else {
      snprintf(buffer, sizeof(buffer), "LL=%02dC  ", LL);
    }

    lcd.print(buffer);

    lcd.setCursor(8, 0);

    dtostrf(temperatura, 4, 1, buffer);

    lcd.print("T=");
    lcd.print(buffer);
    lcd.print("C ");

    lcd.setCursor(0, 1);

    if (modoConfig == 2) {
      snprintf(buffer, sizeof(buffer), "HL=%02d_  ", valorTemporal);
    }
    else {
      snprintf(buffer, sizeof(buffer), "HL=%02dC  ", HL);
    }

    lcd.print(buffer);

    lcd.setCursor(8, 1);

    if (enfriando) {
      lcd.print("[ENFR.] ");
    }
    else {
      lcd.print("[CALEN] ");
    }
  }
}
```

Resultados

Durante las pruebas realizadas se comprobó el funcionamiento del sistema de control de temperatura, observando la respuesta del sensor LM35 y la visualización de los valores en la pantalla LCD. También se verificó el ingreso y la configuración de los límites de temperatura mediante el teclado matricial, así como la respuesta de los actuadores de acuerdo con las condiciones establecidas en el programa. Las siguientes fotografías muestran el montaje físico del circuito y algunas evidencias obtenidas durante las pruebas de funcionamiento.

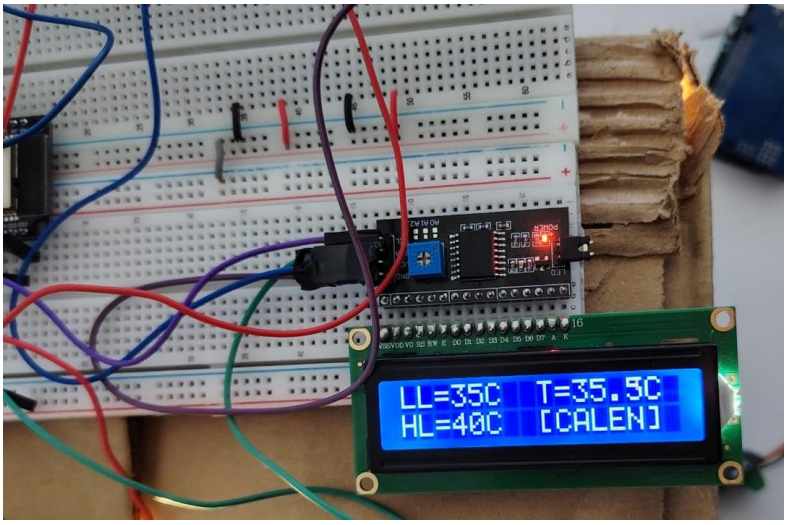


Figura 2. Montaje general del circuito de control de temperatura.

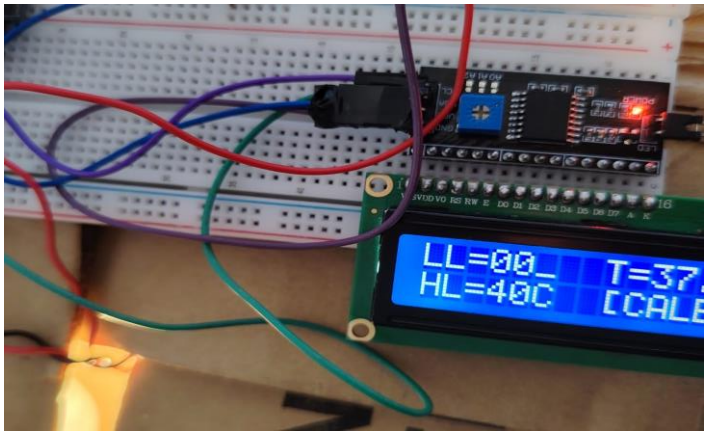


Figura 3. Visualización de la temperatura y los límites configurados en la pantalla LCD.

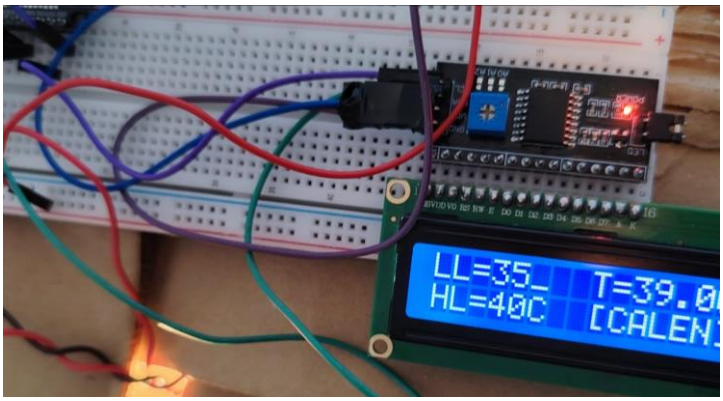


Figura 4. Funcionamiento del sistema con los parámetros de temperatura establecidos.

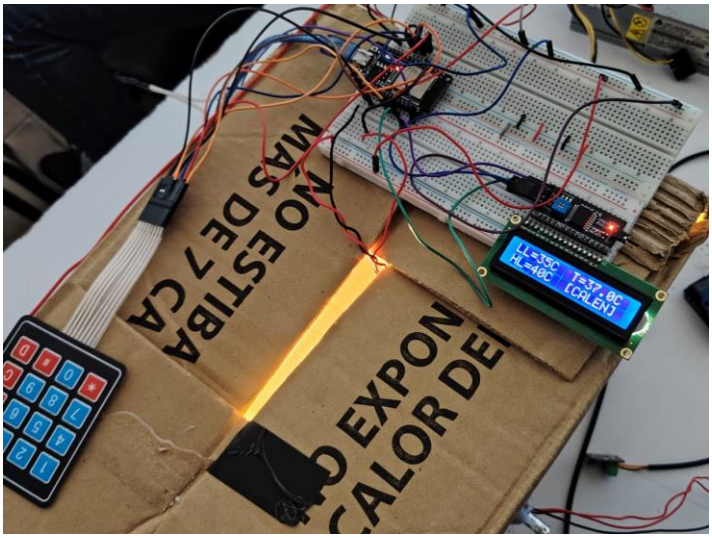


Figura 5. Configuración del límite superior de temperatura mediante el teclado matricial.

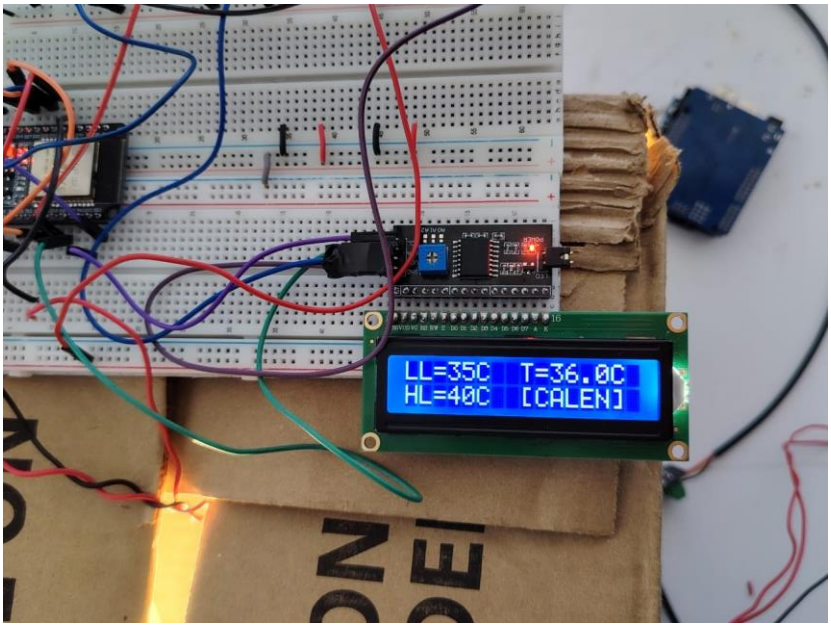


Figura 6. Visualización de los límites de temperatura y lectura del sensor LM35.

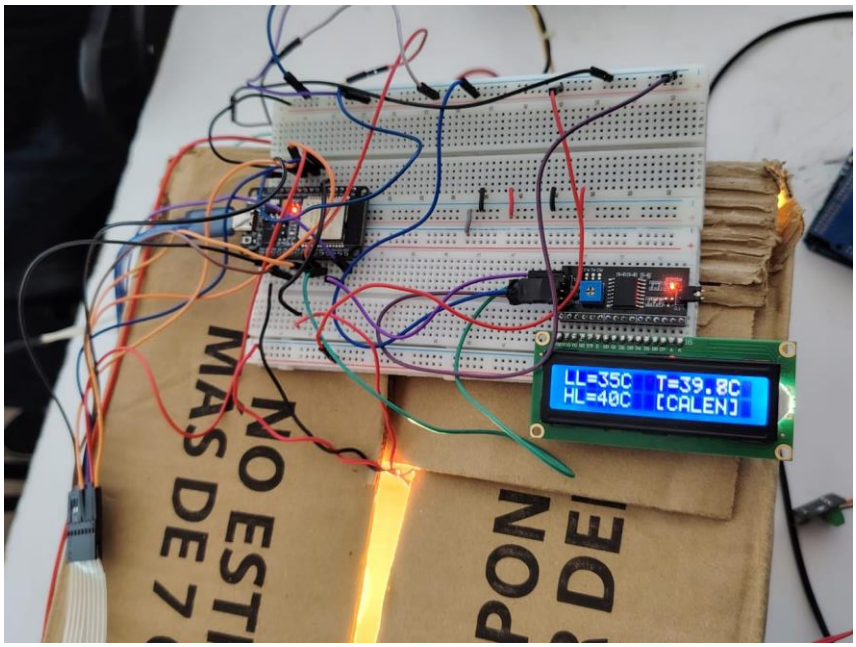


Figura 7. Integración del teclado matricial con el sistema de control.

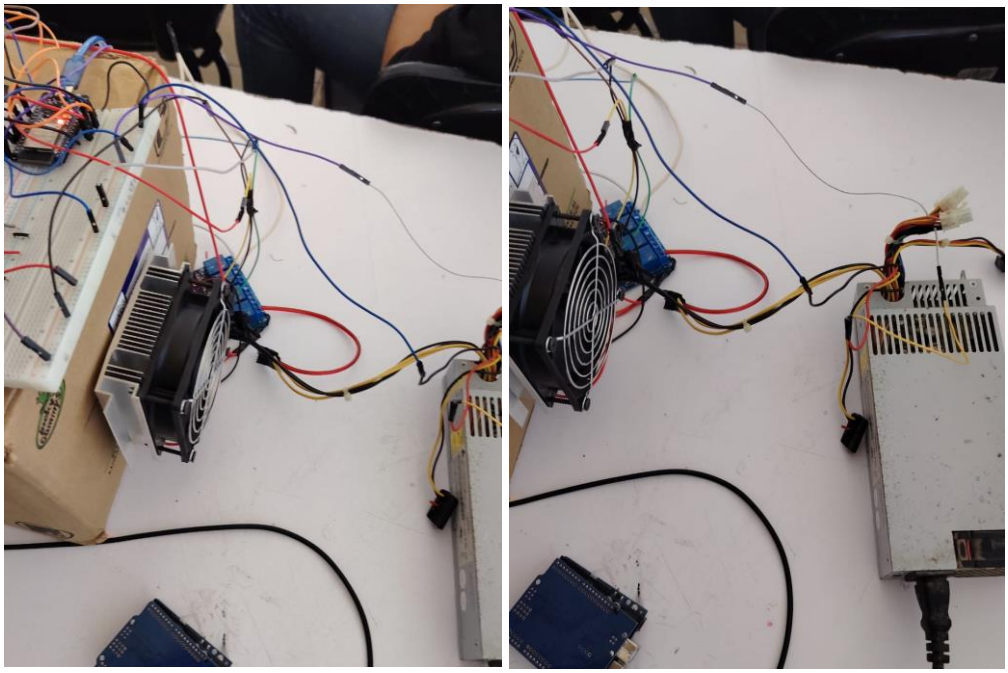


Figura 8. Conexión de los actuadores y elementos de alimentación del sistema.

Observaciones y Conclusiones

Observaciones

Durante el desarrollo de la práctica observamos cómo se pueden integrar diferentes componentes electrónicos para realizar el control de temperatura mediante el ESP32. Al realizar las conexiones y las pruebas, comprobamos que el sensor LM35 permite obtener

la lectura de temperatura y mostrarla en la pantalla LCD, lo que facilita conocer el comportamiento del sistema.

También observamos que el teclado matricial permite modificar los límites de temperatura sin necesidad de cambiar el código del programa. Al ingresar los valores, el sistema verifica que sean correctos y que el límite inferior sea menor que el superior. Otra cosa que pudimos observar fue que, dependiendo de la temperatura registrada, se activan las salidas correspondientes a la calefacción o al enfriamiento.

Durante las pruebas comprobamos la importancia de establecer correctamente las conexiones y revisar la programación para que todos los componentes trabajen de manera adecuada. Además, la pantalla LCD nos permitió visualizar tanto la temperatura como los límites establecidos y el estado en el que se encontraba el sistema.

Conclusiones

Con esta práctica aprendimos a realizar un sistema de control de temperatura utilizando el ESP32, un sensor LM35, un teclado matricial y una pantalla LCD. Comprendimos mejor cómo funciona un control ON-OFF y cómo se utilizan los límites de temperatura para decidir cuándo activar la calefacción o el enfriamiento.

También aprendimos que el teclado matricial puede utilizarse para ingresar y modificar valores, mientras que la pantalla LCD permite mostrar la información del proceso en tiempo real. De igual manera, reforzamos nuestros conocimientos sobre la programación del ESP32, las entradas y salidas digitales y la comunicación entre diferentes componentes electrónicos.

En general, la práctica nos ayudó a comprender cómo se puede automatizar un proceso a partir de la lectura de un sensor y de las condiciones establecidas en un programa. Además, nos permitió relacionar los temas vistos en clase con una aplicación práctica, entendiendo que cada componente cumple una función importante para lograr que el sistema trabaje correctamente.

Referencias

[1] Espressif Systems. (s. f.). ESP32 Series Datasheet. Espressif Systems.

[2] Texas Instruments. (2017). LM35 Precision Centigrade Temperature Sensors. Texas Instruments.

[3] Arduino. (s. f.). Arduino Documentation. Arduino.

[4] International Society of Automation. (s. f.). Temperature Measurement and Control Fundamentals. ISA.

[5] Microchip Technology Inc. (s. f.). Matrix Keypad. Microchip Technology.

[6] NXP Semiconductors. (2021). UM10204: I2C-bus Specification and User Manual. NXP Semiconductors.